

Backdoored Hash Functions: Immunizing HMAC and HKDF

Marc Fischlin Christian Janson Sogol Mazaheri
Cryptoplexity, Technische Universität Darmstadt
Darmstadt, Germany
{marc.fischlin, christian.janson, sogol.mazaheri}@cryptoplexity.de

Abstract—Security of cryptographic schemes is traditionally measured as the inability of resource-constrained adversaries to violate a desired security goal. The security argument usually relies on a sound design of the underlying components. Arguably, one of the most devastating failures of this approach can be observed when considering adversaries such as intelligence agencies that can influence the design, implementation, and standardization of cryptographic primitives. While the most prominent example of cryptographic backdoors is NIST’s Dual_EC_DRBG, believing that such attempts have ended there is naive.

Security of many cryptographic tasks, such as digital signatures, pseudorandom generation, and password protection, crucially relies on the security of hash functions. In this work, we consider the question of how backdoors can endanger security of hash functions and, especially, if and how we can thwart such backdoors. We particularly focus on immunizing arbitrarily backdoored versions of HMAC (RFC 2104) and the hash-based key derivation function HKDF (RFC 5869), which are widely deployed in critical protocols such as TLS. We give evidence that the weak pseudorandomness property of the compression function in the hash function is in fact robust against backdooring. This positive result allows us to build a backdoor-resistant pseudorandom function, i.e., a variant of HMAC, and we show that HKDF can be immunized against backdoors at little cost. Unfortunately, we also argue that safe-guarding unkeyed hash functions against backdoors is presumably hard.

I. INTRODUCTION

The Snowden revelations in 2013 have exposed several ongoing surveillance programs targeting people all over the world, violating their privacy, and endangering their security [5], [27]. Different techniques have been used from installing backdoors, injecting malware, and undermining standardization processes to simply name a few. A prominent example is NIST’s pseudorandom generator Dual_EC_DRBG, which is widely believed to have been backdoored by the National Security Agency (NSA) [6], [15]. An entity choosing the elliptic curve parameters used in Dual_EC_DRBG can not only distinguish the outputs of the pseudorandom generator (PRG) from random but also predict future outputs.

Studying deliberate and covert weakening of cryptosystems by embedding backdoors in primitives and subverting implementations was initiated already over two decades ago by Young and Yung [42], [43] in a line of work referred to as *kleptography*. Recent revelations have drawn our community’s

attention more than ever before to the realness and the gravity of such attacks and the increasing importance of their rigorous treatment (cf. Section VIII for related work).

In this work we turn our attention to understanding and immunizing *backdoored hash functions*. A hash function is a function that compresses an arbitrary-length input to a short, fixed-length output. To name a few applications, hash functions are used in message authentication codes (MACs) such as HMAC (RFC 2104), signature schemes, pseudorandom generation, randomness extraction such as HKDF (RFC 5869), and password protection. The security of these applications among others relies crucially on the security of the underlying hash function. Naturally, if the employed hash function is backdoored by its malicious designer, all bets on the standard security guarantees are off.

We believe that studying the impact of backdoored cryptographic primitives on their applications and developing strategies to build backdoor-resistant constructions is of utmost necessity. Unfortunately, immunizing hash functions against backdoors and reviving their security against the backdooring adversary is far from easy. Most repellents against backdoors in cryptographic primitives are cumbersome, and often require additional means like reliable alternative primitives or complex detection mechanisms. Interestingly, we argue here that lightweight immunization of hash-based MACs (namely, HMAC) and hash-based key derivation functions (namely, HKDF) is possible.

A. Our Results

BACKDOORED HASH FUNCTIONS. Our work begins with formalizing backdoored hash functions. A backdooring adversary generates a backdoored hash function family together with a short bit string corresponding to a backdoor key. The adversarial influence in the design is captured by this definition, since the hash function family and its constants can be chosen maliciously in a way that a short backdoor key co-designed with the family enables bypassing the security guarantees. We revisit the main security requirements, i.e., collision resistance, preimage resistance, and second-preimage resistance to include the possibility of backdoored hash function generation. Intuitively, a backdoored hash function retains security against adversaries that do not hold a backdoor key, since a rational malicious designer would want the backdoor to be exclusive.

This work has been supported by the German Federal Ministry of Education and Research (BMBF) as well as by the Hessian State Ministry for Higher Education, Research and the Arts, within CRISP.

HOW TO IMMUNIZE. Withstanding backdoor attacks is hard. Therefore, we pursue the quest of identifying cryptographic properties that provide a successful immunization against all types of backdoors in hash functions. Fortunately, we are able to identify a promising candidate property of the compression function which cannot be weakened by a backdoor. This property is *weak pseudorandomness*, saying that the compression function's outputs on random inputs look random. The reason for our optimism is that we can show that distinguishing the outputs (on random inputs) from random *with a backdoor* implies public-key encryption. In other words, placing a backdoor in the hash function design implicitly needs to embed a tedious public-key scheme and makes the design look suspicious. Hence, unless there is surprising progress in the efficiency of public-key schemes, fast compression functions will not be built from public-key tools and hence will remain weakly pseudorandom, even with knowledge of the backdoor. This result follows an idea of Pietrzak and Sjödin [38] for building key agreement from secret-coin weak pseudorandom functions.

Using the assumption of weak pseudorandomness we are able to provide an immunization strategy for HMAC based on the randomized cascade construction introduced by Maurer and Tessaro [33]. On a high level, the construction makes use of a prefix-free encoding to map blocks of the input message to (honestly chosen) random strings. We argue that since the randomized cascade construction yields a pseudorandom function (PRF), it can be used in the inner HMAC chain showing that such a modified HMAC is a PRF. However, there is a small caveat in terms of efficiency since the underlying transformation in the randomized cascade construction from a weak PRF to a full-fledged PRF can be expensive in terms of the number of compression function evaluations.

We further investigate whether there exist simpler immunization solutions (compared to the randomized cascade construction) for *key derivation functions* based on hash functions, especially HKDF based on HMAC. Fortunately, we answer this in the affirmative and show that an idea by Halevi and Krawczyk [28] for strengthening hash-and-sign schemes via input randomization can be used to immunize HMAC when used as a key derivation function. The result again relies on the weak pseudorandomness of the compression function.

BACKDOORED CONSTRUCTION. We finally demonstrate the feasibility of embedding a backdoor in a hash function by constructing a backdoored Merkle-Damgård-based hash function, which iterates a backdoored compression function. One may think that building a backdoored hash function, which is secure without the backdoor but insecure with the backdoor key would imply public-key encryption (or equivalently trapdoor permutations), as it does in case of backdoored *weak* pseudorandom functions (as mentioned above) and backdoored PRGs [21]. We show, however, that for unkeyed (aka., publicly keyed) hash functions and (strong) pseudorandom functions this is unfortunately not necessarily true. They can in principle be as fast as an unbackdoored one. Our construction is

inspired by many-to-one trapdoor one-way functions with an exponential preimage size as studied by Bellare et al. [11].

On a high level, a malicious designer can build a backdoored compression function from an arbitrary secure compression function, where the backdoored function basically “mimics” the behavior of the healthy function unless a backdoor key, which is a particular bit string, is given as part of the input. For this exceptional input the altered function returns something trivial, e.g., a part of its input. In other words, the backdoor key acts as a *logical bomb* for the backdoored compression function, thereby triggering a malicious behavior. Since the inputs to hash functions can be chosen by the adversary, finding collisions, preimages, and second preimages becomes easy when triggering the backdoor. It is noteworthy that the backdoor can only be triggered by an adversary with prior knowledge of the backdoor key, since it is cryptographically hidden in the construction.

Furthermore, we show that even though HMAC uses a secret key, it is not secure against an adversary that can exploit the backdoor for the underlying hash function. This enables the adversary to find collisions in the inner hash chain which is exactly what makes forging MAC tags possible.

B. Structure of the Paper

In Section II, we provide a formal definition of backdoored hash functions and establish security notions for standard and backdoored hash functions. In Section III, we show that backdoored weak pseudorandom functions imply public-key encryption. Based on this positive result, we provide a solution for immunizing backdoored HMAC constructions in Section IV and give a more efficient solution for immunizing HKDF in Section V. We discuss applications to the pre-shared key mode of the TLS 1.3 handshake protocol candidate in Section VI. In Section VII, we concretely show that security of a Merkle-Damgård-based backdoored hash function and HMAC can unfortunately be completely undermined if the iterated compression function is backdoored. In Section VIII, we discuss related work and finally we conclude the paper in Section IX.

II. MODELING BACKDOORED HASH FUNCTIONS

In this section we give some background on hash functions and their security, recall the Merkle-Damgård transform for building hash functions from fixed input-length compression functions as well as the HMAC construction. Finally, we give a formal definition of backdoored hash functions and extend standard security notions to additionally capture security against the backdooring adversary.

NOTATION. The set of bit strings of arbitrary length is denoted by $\{0,1\}^*$ and the set of bit strings of length at most n by $\{0,1\}^{\leq n}$. The length of a bit string $s \in \{0,1\}^*$ is denoted by $|s|$ and a special symbol ϵ indicates a string of length 0. The concatenation of two bit strings s_1 and s_2 by $s_1||s_2$. By $s_{[i,j]}$ we denote the substring of s starting from the i -th bit and ending with the j -th bit, where the first index of a string is $i = 0$. We write $s \xleftarrow{\$} S$ to denote the sampling of a

value uniformly at random from a finite set S . We use PPT to denote probabilistic polynomial-time and denote by $\text{poly}(\lambda)$ an unspecified polynomial in the security parameter. For a run of a randomized algorithm A on input x and with randomness r we write $A(x; r)$. Accordingly, for a probabilistic algorithm A the random variable $A(x)$ describes its output, and we write $y \xleftarrow{\$} A(x)$ for the sampling. For a deterministic algorithm we simply write $y \leftarrow A(x)$. An optional input value x for an algorithm is put in square brackets $[x]$. It is sometimes convenient to write $[x]_b$ to make the presence of the optional input dependent on a bit b , i.e., $A([x]_b)$ means that A receives the input if $b = 1$, and not if $b = 0$.

A. Hash Functions

Informally, a hash function is an efficiently computable function which compresses bit strings of arbitrary length to bit strings of a fixed length. Inputs of hash functions are often referred to as *messages* and their outputs are often called *digests*. Depending on concrete applications, cryptographic hash functions are required to meet certain security requirements, among which *collision resistance*, *preimage resistance*, and *second-preimage resistance* are the most common ones. Roughly speaking, collision resistance means that it is computationally infeasible to find any two distinct messages which will be mapped to the same digest. Preimage resistance, also known as one-wayness, concerns the infeasibility of finding a message that hashes to a given random digest of the hash function. Finally, second-preimage resistance indicates that given a random message it is computationally infeasible to find a second distinct message that collides with the given message. We formalize the above security notions later in this section.

To bridge the gap between keyed hash functions in theory and unkeyed hash functions in practice, we adopt the more general notion of hash functions as families of keyed functions. The keys are public such that a key here can be thought of as an index specifying which particular hash function from the family is being considered. For unkeyed hash functions the key can be set to some constant.

Definition II.1 (Hash Function). *A hash function is a pair of efficient algorithms $\mathcal{H} = (\text{KGen}, H)$ with associated key space $\mathcal{K}$, message space $\mathcal{M}$, and digest space $\mathcal{D}$, such that:*

- $k \xleftarrow{\$} \text{KGen}(1^\lambda)$: *On input of a security parameter, this probabilistic polynomial-time algorithm generates and outputs a key $k \in \mathcal{K}$;*
- $d \leftarrow H(k, m)$: *On input of a key $k \in \mathcal{K}$ and a message $m \in \mathcal{M}$, this deterministic polynomial-time algorithm outputs a digest $d \in \mathcal{D}$.*

We write $H_k(m)$ as a shorthand for $H(k, m)$. Since the key is public, it is often helpful to identify it as an initialization vector in concrete constructions of hash functions, denoted by IV.

1) *Merkle-Damgård-based Hash Functions*: The Merkle-Damgård construction [18], [34] is one of the most commonly used approaches for building a full-fledged hash function with

arbitrary input length. The construction works by iterating a compression function, processing a single block of the input message in each iteration and using padding techniques to make the entire input message length comply with the block length. In terms of security, for appropriate paddings it preserves the collision resistance of the iterated compression function. The Merkle-Damgård domain extender is extensively used in practice for hash functions including the MD family, SHA-1 and SHA-2, while each one employs a different compression function.

In the following, we describe a generic Merkle-Damgård-based hash function $\mathcal{H}_h^{\text{md}} := (\text{KGen}^{\text{md}}, H_h^{\text{md}})$ with associated key space $\mathcal{K} := \{0, 1\}^\ell$, message space $\mathcal{M} := \{0, 1\}^{\leq 2^p}$ for some fixed integer p , and digest space $\mathcal{D} := \{0, 1\}^\ell$, iterating a compression function $h: \{0, 1\}^\ell \times \{0, 1\}^b \rightarrow \{0, 1\}^\ell$. As described in Figure 1, an input message m is first padded such that its length becomes a multiple of the block size b that is processable by the compression function. The padded message is then split into blocks $m_0, m_1, \dots, m_{n-1}$, where each message block is of size b . Below we discuss the padding function in more detail. Next the compression function h is iterated in such a way that the output of the previous compression function and the next message block become the input to the next compression function. The iteration starts with an initialization value $\text{IV} \xleftarrow{\$} \text{KGen}^{\text{md}}(1^\lambda)$ and the first message block m_0 .

```

 $\mathcal{H}_{h, \text{IV}}^{\text{md}}(m)$ 
 $m \leftarrow m || \text{lpad}(m, b, p)$ 
parse  $m$  as  $m_0 || \dots || m_{n-1}$  where  $|m_i| = b$  for all  $0 \leq i < n$ 
 $d_0 \leftarrow \text{IV}$ 
for  $i = 0 \dots n - 1$  do
     $d_{i+1} \leftarrow h(d_i, m_i)$ 
return  $d_n$ 

```

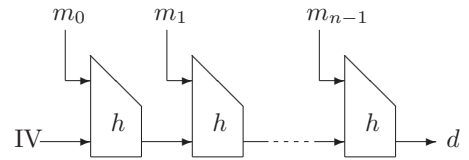


Figure 1: Merkle-Damgård construction from a compression function h .

LENGTH PADDING. The padding used in the domain extender must itself be collision free. Length padding is typically used for Merkle-Damgård-based hash functions. It appends the length of the message to the end, while making sure that the length of the padded message is a multiple of the block size b required by the compression function. We consider a compact length padding function lpad that uses p bits to represent the message length, where p is usually smaller than or equal to ℓ (e.g., $p = 64$ for SHA-256). Hence, the padded message contains the message length in its last b -bits block possibly together with some of the least significant bits of the message.

Such a length padding function is commonly used in practical Merkle-Damgård-based hash functions, such as MD5, SHA-1 and SHA-2. A similar padding that additionally prepends the length of the message with a bit of 1 is used in BLAKE. Let $\text{binary}(x, y)$ be the binary representation of x in y bits, then lpad is defined as:

$$\text{lpad}(m, b, p) := 1 || 0^{(b-|m|-p-1) \bmod b} || \text{binary}(|m|, p).$$

A less compact and mostly theoretical variant of length padding uses exactly b bits for representing the message length, which is then encoded in a separate block.

2) *The HMAC Scheme*: Message authentication codes (MACs) provide message integrity, i.e., they can be used to prevent adversaries from tampering with a communication without being detected by the receiver. The widely used HMAC scheme [9] is built on a cryptographic hash function. It has been standardized in IETF RFC [30] and NIST [24], and is widely deployed in various security protocols such as for example TLS and IPsec. HMAC (to be precise, its theoretical counterpart NMAC) is provably a pseudorandom function, i.e., indistinguishable from a random function, under the assumption that its underlying compression function is a pseudorandom function [8]. Note that PRF security implies the standard notion of unforgeability for MAC schemes.

Definition II.2 (HMAC). Let $\mathcal{H}_h^{md} = (\text{KGen}^{md}, H_h^{md})$ be a Merkle-Damgård-based hash function with associated key space $\mathcal{K}$, message space $\mathcal{M}$, and digest space $\mathcal{D}$. The hash-based message authentication scheme $\text{HMAC}_h = (\text{KGen}, \text{HMAC}_h)$ with associated secret key space $\mathcal{SK}$, message space $\mathcal{M}$, and tag space $\mathcal{T}$ is defined as:

- $(k, \text{IV}) \xleftarrow{\$} \text{KGen}(1^\lambda)$: On input of a security parameter, this PPT algorithm outputs a secret key $k \in \mathcal{SK}$ and an initial value $\text{IV} \in \mathcal{K}$, where $\text{IV} \xleftarrow{\$} \text{KGen}^{md}(1^\lambda)$.
- $t \leftarrow \text{HMAC}_h(k, \text{IV}, m)$: On input of a key $k \in \mathcal{SK}$, an initial value $\text{IV} \in \mathcal{K}$, and a message $m \in \mathcal{M}$, this deterministic polynomial-time algorithm outputs a tag $t \in \mathcal{T}$:

$$\text{HMAC}_h(k, \text{IV}, m) = H_{h, \text{IV}}^{md}((k \oplus \text{opad}) || H_{h, \text{IV}}^{md}((k \oplus \text{ipad}) || m)),$$

where ipad and opad are fixed, distinct b -bit constants.

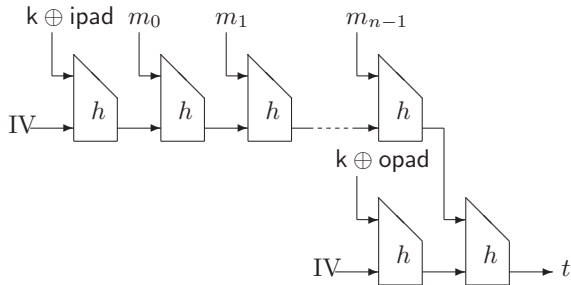


Figure 2: Illustration of HMAC.

B. Backdoored Hash Functions

A backdoored hash function is a function which is designed by an adversary together with a short backdoor key, whose knowledge allows for violating the security of the hash function. More precisely, we consider an efficient algorithm BDHGen , which outputs a hash function family $\mathcal{H}$, a backdoor bk , and a string r . The latter, if not empty, is used as the randomness in key generation. Otherwise, the key generation algorithm uses its own random coins. The algorithm BDHGen can be seen as a designer of a hash function $\mathcal{H}$ installing a backdoor bk , potentially depending (via r) on a specific instance of the hash function family.

Definition II.3 (Hash Function Generator). A PPT algorithm BDHGen is called a hash function generator, if on input of a security parameter 1^λ , it outputs (the description of) a hash function family $\mathcal{H}$, a backdoor key $\text{bk} \in \{0, 1\}^{\text{poly}(\lambda)}$, and a potentially empty randomness string $r \in \{0, 1\}^{\text{poly}(\lambda)}$.

Before formally defining backdoored hash functions, we give definitions for the most commonly used security notions of hash functions. We deviate slightly from the classic formulations and generate the key with adversarial chosen randomness (if provided by BDHGen). Moreover, we optionally give the adversary the backdoor key, the availability depending on a bit b . Our definitions are thus general enough to capture standard security notions without backdoors ($b = 0$) as well as security against backdoored hash functions ($b = 1$), both with and without influencing the key generation.

Definition II.4 (Collision Resistance). The advantage of an adversary $\mathcal{A}$ in finding collisions for a hash function generated by a PPT hash function generator BDHGen is defined below. The bit b indicates whether the adversary knows the generated backdoor key.

$$\text{Adv}_{\text{BDHGen}, \mathcal{A}, b}^{\text{CR}}(\lambda) := \Pr \left[\begin{array}{l} H_k(m) = H_k(m') \\ \wedge m \neq m' \end{array} \mid \begin{array}{l} (\mathcal{H}, \text{bk}, r) \xleftarrow{\$} \text{BDHGen}(1^\lambda) \\ \wedge k \xleftarrow{\$} \text{KGen}(1^\lambda; r) \\ \wedge (m, m') \xleftarrow{\$} \mathcal{A}([\text{bk}]_b, k) \end{array} \right],$$

where the probability is over the internal random coins of BDHGen , $\mathcal{A}$, and potentially KGen . We call BDHGen collision resistant (resp. collision resistant with backdoor) if for all PPT adversaries $\mathcal{A}$, the above advantage for $b = 0$ (resp. $b = 1$) is negligible.

Definitions of preimage resistance and second-preimage resistance are parameterized with an integer n which indicates the length of the challenge message. Doing so allows us to easily circumvent the technicality of uniformly sampling from an infinite set.

Definition II.5 (Preimage Resistance). Let n be an integer such that $\{0, 1\}^n \subseteq \mathcal{M}$. The advantage of an adversary $\mathcal{A}$ in finding preimages for a hash function generated by a PPT hash function generator BDHGen is defined below. The bit b

indicates whether the adversary knows the generated backdoor key.

$$\text{Adv}_{\text{BDHGen}, \mathcal{A}, b}^{\text{PR}, n}(\lambda) := \Pr \left[\begin{array}{l} \mathcal{H}_k(m') = \mathcal{H}_k(m) \\ \wedge m \leftarrow \{0, 1\}^n \\ \wedge m' \leftarrow \mathcal{A}([\text{bk}]_b, \mathcal{H}_k(m), k) \end{array} \mid \begin{array}{l} (\mathcal{H}, \text{bk}, r) \leftarrow \text{BDHGen}(1^\lambda) \\ \wedge k \leftarrow \text{KGen}(1^\lambda; r) \end{array} \right],$$

where the probability is over the random choice of m and the internal random coins of BDHGen , $\mathcal{A}$, and potentially KGen . We call BDHGen preimage resistant or one-way (resp. preimage resistant with backdoor) for parameter n if for all PPT adversaries $\mathcal{A}$ the above advantage for $b = 0$ (resp. $b = 1$) is negligible.

Definition II.6 (Second-Preimage Resistance). Let n be an integer such that $\{0, 1\}^n \subseteq \mathcal{M}$. The advantage of an adversary $\mathcal{A}$ in finding second preimages for a hash function generated by a PPT hash function generator BDHGen is defined below. The bit b indicates whether the adversary knows the generated backdoor key.

$$\text{Adv}_{\text{BDHGen}, \mathcal{A}, b}^{\text{SPR}, n}(\lambda) := \Pr \left[\begin{array}{l} \mathcal{H}_k(m') = \mathcal{H}_k(m) \\ \wedge m \neq m' \end{array} \mid \begin{array}{l} (\mathcal{H}, \text{bk}, r) \leftarrow \text{BDHGen}(1^\lambda) \\ \wedge k \leftarrow \text{KGen}(1^\lambda; r) \\ \wedge m \leftarrow \{0, 1\}^n \\ \wedge m' \leftarrow \mathcal{A}([\text{bk}]_b, m, k) \end{array} \right],$$

where the probability is over the random choice of m , and the internal random coins of BDHGen , $\mathcal{A}$, and potentially KGen . We call BDHGen second-preimage resistant (resp. second-preimage resistant with backdoor) for parameter n if for all PPT adversaries $\mathcal{A}$, the above advantage for $b = 0$ (resp. $b = 1$) is negligible.

Definition II.7 (Backdoored Hash Function). Let BDHGen be a PPT hash function generator and $S \in \{\text{CR}, \text{PR}, \text{SPR}\}$ denote a security notion for hash functions.

We call BDHGen a backdoored hash function generator (and its output hash function a backdoored hash function), if there is a PPT adversary $\mathcal{A}$ such that the advantage $\text{Adv}_{\text{BDHGen}, \mathcal{A}, 1}^{S, [n]}(\lambda)$ is non-negligible.

We call BDHGen a weakly backdoored hash function generator (and its output hash function a weakly backdoored hash function) if the randomness string r output by BDHGen is not empty. At the same time, however, for all PPT adversaries $\mathcal{A}$ without bk the advantage $\text{Adv}_{\text{BDHGen}, \mathcal{A}, 0}^{S, [n]}(\lambda)$ is still negligible.

Weakly backdoored hash functions only provide a backdoor if the key generation algorithm is run on the randomness r . They are defined for the sake of completeness, since hash functions are usually used with fixed keys in practice. However, throughout this paper we consider the stronger notion of backdoors, which allow attacks for randomly chosen keys. In this case, the malicious designer of the hash function does not need to influence the hash key generation, i.e., r is empty.

III. ON THE IMPLAUSIBILITY OF BACKDOORED WEAK PSEUDORANDOM FUNCTIONS

We argue that it is reasonable to assume that a backdoored weak PRF, which is secure in the standard sense against distinguishers who do not know the backdoor, remains a weak PRF even against distinguishers who know the backdoor. We prove that if a backdoor allows for distinguishing outputs of a weak PRF on random inputs from uniform random bit strings, then that weak PRF family already implies public-key encryption. Put differently, any such backdoored function would need to already contain some form of public-key encryption. Such systems, however, are significantly slower than symmetric-key based pseudorandom functions.

A. Weak Pseudorandom Functions

A family of functions $f: \{0, 1\}^k \times \{0, 1\}^i \rightarrow \{0, 1\}^o$ is called *weakly pseudorandom* if no efficient adversary can distinguish a random function of the family from a uniform random function when queried on random inputs. More precisely, let a family of functions, and potentially a backdoor, be generated by a PPT generator $(f, \text{bk}) \leftarrow \text{BDPRFGen}(1^\lambda)$. An adversary attacking the weak pseudorandomness of f can only query an oracle on an integer $q \leq \text{poly}(\lambda)$, where the oracle either implements a function $F(k, \cdot)$ or a random function $F_\$(\cdot)$. Upon being queried on q , the oracle outputs q input-output pairs (x, y) such that $x \leftarrow \{0, 1\}^i$ is random and $y = f(k, x)$ (resp. $y = f_\$(x)$). Here, $k \leftarrow \{0, 1\}^k$ is also chosen at random (resp. $f_\$ \leftarrow \text{Func}(i, o)$ is a randomly chosen function from the set $\text{Func}(i, o)$ of all functions from $\{0, 1\}^i$ to $\{0, 1\}^o$). The weak PRF-advantage of an adversary $\mathcal{A}$, optionally using the backdoor bk (based on a bit b), is then defined by:

$$\begin{aligned} \text{Adv}_{\text{BDPRFGen}, \mathcal{A}, b}^{w\text{PRF}}(\lambda) := & \left| \Pr \left[\mathcal{A}^{F(k, \cdot)}(1^\lambda, [\text{bk}]_b) = 1 \mid \right. \right. \\ & (f, \text{bk}) \leftarrow \text{BDPRFGen}(1^\lambda), k \leftarrow \{0, 1\}^k \\ & \left. \left. - \Pr \left[\mathcal{A}^{F_\$(\cdot)}(1^\lambda, [\text{bk}]_b) = 1 \mid \right. \right. \right. \\ & \left. \left. (f, \text{bk}) \leftarrow \text{BDPRFGen}(1^\lambda), f_\$ \leftarrow \text{Func}(i, o) \right] \right|, \end{aligned}$$

where the probability is over the choice of $f_\$$ resp. k , BDPRFGen and $\mathcal{A}$'s coin tosses.

Remark. When saying that a compression function $h: \{0, 1\}^\ell \times \{0, 1\}^b \rightarrow \{0, 1\}^\ell$ is weakly pseudorandom we mean that the key k is chosen from $\{0, 1\}^\ell$ and that we consider the function $h(k, \cdot): \{0, 1\}^b \rightarrow \{0, 1\}^\ell$.

Remark. Note that if the backdoor key is not given to the adversary, we obtain the standard security notion for weak PRFs. We say that f is a *backdoored weak PRF* if f is a weak PRF against adversaries without the backdoor, while with the backdoor there exists a PPT distinguisher $\mathcal{A}$ against its weak pseudorandomness that has a non-negligible advantage.

B. Backdoored Weak PRFs Imply Public-Key Encryption

In the following we construct a public-key scheme, given a backdoored weak PRF. The idea for this constructions is based

on the work of Pietrzak and Sjödin [38] for building key agreement from secret-coin weak pseudorandom functions. Since the backdoor allows to distinguish pseudorandom from random strings, even on random inputs, we can use the backdoor to decrypt bit encryptions. The sender encodes the bit b to be sent by using pseudorandom answers $y_j = f(x_j)$ for random inputs x_j for $b = 1$, and truly random answers y_j instead to encode the bit $b = 0$. The backdoor holder can then distinguish the two cases and hence recover the bit with some probability bounded away from $\frac{1}{2}$ (and we can amplify the success probability via repetitions). Since one cannot distinguish random outputs of a weak PRF from uniform outputs without the backdoor, we obtain a secure public-key encryption scheme.

We give the construction and proof in terms of concrete security but occasionally refer to the common asymptotic setting. As for asymptotic behavior, we note that we get an infinite-often public-key encryption scheme, where infinitely often means that the decryption algorithm works for infinitely many security parameters.

Theorem III.1. *Let $f : \{0, 1\}^k \times \{0, 1\}^i \rightarrow \{0, 1\}^o$ be a backdoored weak pseudorandom function. Then we can build an IND-CPA-secure public-key bit encryption scheme from f .*

Proof. Let BDPRFGen be a generator for a backdoored weak pseudorandom function family. Suppose $\mathcal{A}$ is a PPT adversary such that $\mathcal{A}$'s only query to its oracle is on an integer $q \leq \text{poly}(\lambda)$ and it holds that $\text{Adv}_{\text{BDPRFGen}, \mathcal{A}, 1}^{w\text{PRF}}(\lambda) = \varepsilon \not\approx 0$. In particular, $\varepsilon \geq \frac{1}{\text{poly}(\lambda)}$ infinitely often.

Then we can construct a public-key bit encryption scheme $\mathcal{E} := (\text{KGen}, \text{Enc}, \text{Dec})$ with overwhelming correctness as follows. For sake of simplicity we first construct an encryption scheme with correctness $\frac{1}{2} + \varepsilon$ and explain afterwards how to boost the correctness bound.

$\text{KGen}(1^\lambda)$ $(f, \text{bk}) \xleftarrow{\$} \text{BDPRFGen}(1^\lambda)$ $\text{pk} \leftarrow f$ $\text{sk} \leftarrow \text{bk}$ return (pk, sk)	$\text{Enc}(\text{pk}, b)$ if $b = 0$ then $c \xleftarrow{\$} \{0, 1\}^{(i+o) \cdot q}$ else $k \xleftarrow{\$} \{0, 1\}^k, c \leftarrow \epsilon$ for $j = 1 \dots q$ do $x_j \xleftarrow{\$} \{0, 1\}^i, y_j \leftarrow f(k, x_j)$ $c \leftarrow c x_j y_j$ return c
$\text{Dec}(\text{sk}, c)$ $b \xleftarrow{\$} \mathcal{A}(\text{sk}, c)$ return b	

For correctness observe that by construction a ciphertext can be correctly decrypted if $\mathcal{A}$ successfully distinguishes random outputs of $f(k, \cdot)$ from uniformly random bit strings. Hence we obtain the following correctness:

$$\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, b)) = b] = \Pr[\mathcal{A}(\text{sk}, \text{Enc}(\text{pk}, b)) = b] = \frac{1}{2} + \text{Adv}_{\text{BDPRFGen}, \mathcal{A}, 1}^{w\text{PRF}}(\lambda) = \frac{1}{2} + \varepsilon,$$

In other words, the decryption error is noticeably smaller than $\frac{1}{2}$ for infinitely many security parameters.

It remains to show that $\mathcal{E}$ is indistinguishable under chosen-plaintext attacks. Suppose in the contrary that there exists

a PPT adversary $\mathcal{B}$ against the security of $\mathcal{E}$. Since we are concerned with security of bit encryption, this means that $\mathcal{B}$ can decrypt a ciphertext with a non-negligible advantage ε' . We can build from $\mathcal{B}$ a PPT adversary $\mathcal{C}$ against the weak PRF security of BDPRFGen, i.e., the generated family f (when not holding a backdoor). When $\mathcal{C}$ queries its oracle on an integer q , it obtains a string c containing q pairs of random messages with the result of the oracle evaluation on them. It then runs $\mathcal{B}$ on that value c . When $\mathcal{B}$ finally terminates with output bit b , the adversary $\mathcal{C}$ also outputs b as its guess. We obtain:

$$\text{Adv}_{\text{BDPRFGen}, \mathcal{C}, 0}^{w\text{PRF}}(\lambda) = \text{Adv}_{\mathcal{E}, \mathcal{B}}^{\text{IND-CPA}}(\lambda) = \Pr[\mathcal{B}(\text{pk}, \text{Enc}(\text{pk}, b)) = b] - \frac{1}{2} = \varepsilon'.$$

Thus our adversary $\mathcal{C}$, who does not know a backdoor key, has a non-negligible advantage against the weak pseudorandomness of f . This contradicts our assumption of f being weakly pseudorandom (without the backdoor key).

The final step is to note that we can reduce the decryption error by standard techniques. For this we repeat the basic encryption step above polynomial times, letting the sender always generate pseudorandom ($b = 1$) or truly random strings ($b = 0$) in each of the repetitions. The decrypter outputs a majority decision for all the extracted bits. If we use $\lambda \cdot \varepsilon^2 \leq \lambda \cdot \text{poly}(\lambda)^2$ repetitions, where $\varepsilon \geq \frac{1}{\text{poly}(\lambda)}$ is the lower bound for the distinguisher's advantage, then the Hoeffding-Chernoff bound implies that the decryption error is lower-bounded by $e^{-\lambda}$. At the same time, the security of the public-key encryption scheme remains intact for a polynomial number of repetitions. $\square$

IV. IMMUNIZATION OF HMAC

According to the result presented in the previous section, we may assume that the compression function used in an HMAC construction preserves weak pseudorandomness in the presence of backdoors. In the following we use the randomized cascade (RC) construction introduced by Maurer and Tessaro [33] in order to immunize HMAC under the weak pseudorandomness assumption. In basic terms, the RC construction is an iterated construction of a PRF from a (constant-query) weak PRF. The first construction of a PRF from a weak PRF is due to Naor and Reingold [37], and a further construction was later proposed by Maurer and Sjödin [32]. In our case, we are interested in an iterated construction of a PRF where the candidate for a weak PRF may be a compression function of hash functions. Maurer and Tessaro note that both constructions [32], [37] may be turned easily into iterative versions with the drawback that the number of calls to the function would increase significantly. In contrast, the randomized cascade construction is more efficient and requires for input length b approximately $\frac{b}{\log s}$ (for $s \geq 2$) many calls to the function and also only requires the weaker

underlying assumption of an s -query weak PRF¹ than weak PRF.

Let us now review the idea of the RC construction which itself is based on the cascade construction for hash functions by Bellare, Canetti and Krawczyk [10]. The RC construction requires a prefix-free encoding of the input (from some set $\mathcal{X}$). We say an efficiently computable encoding $\text{Encode}: \mathcal{X} \rightarrow \{1, \dots, s\}^+$ is prefix-free if for all distinct inputs $x, x' \in \mathcal{X}$ the sequence $\text{Encode}(x)$ is not a prefix of the sequence $\text{Encode}(x')$. On a high level, the RC construction follows the principles of the Merkle-Damgård construction (cf. Figure 1) with some additional randomness where the underlying building block is a s -weak PRF.

The RC construction with parameter s and input set $\mathcal{X} \supseteq \mathcal{M}$ for the function $h: \{0, 1\}^\ell \times \{0, 1\}^b \rightarrow \{0, 1\}^\ell$ and a prefix-free encoding as described above is a mapping $\text{RC}_{s, \mathcal{X}, \text{Encode}}^h: \{0, 1\}^\ell \times \{0, 1\}^{s \cdot b} \times \mathcal{X} \rightarrow \{0, 1\}^\ell$. The mapping uses as input a private key k of length ℓ and a $(s \cdot b)$ -bit long public part which can be interpreted as the concatenation of s b -bit strings $r_1, \dots, r_s$ and an input $x \in \mathcal{X}$. The input x is first padded (following Section VII) such that the length becomes a multiple of b and is then further processed with the above prefix-free encoding outputting a sequence $(m_1, \dots, m_n) \in \{1, \dots, s\}^+$. Then for $i = 1, \dots, n$ the cascade is computed as $y_{i+1} \leftarrow h(y_i, r_{m_i})$ with $y_1 \leftarrow k$. First let us remark that in each iteration the r_{m_i} 's are chosen according to the outputted sequence from the encoding. Maurer and Tessaro formally prove given that h is a s -weak PRF then the resulting RC construction is a PRF. The proof relies on the encoding being done via a tree structure, where it is argued that by the definition of s -weak PRF whenever we evaluate the function under some secret key at s independent random inputs, it produces s pseudorandom outputs and in particular sets all vertices in the tree to be pseudorandom.

Now let $\text{HMAC}_h := (\text{KGen}, \text{HMAC}_h)$ be a backdoored HMAC construction. Our goal is to replace the Merkle-Damgård construction with the randomized cascade construction from above and argue that one can salvage HMAC in the context of backdoored hash functions. The idea is that we first pad the input message m with the usual length padding function and then use a prefix-free encoding obtaining a sequence m' . According to the sequence the correct random string will be chosen and used as the input to the compression function. More formally it follows that

$$\text{HMAC}_h^{\text{rc}}(k, \text{IV}, m) = \text{H}_{h, \text{IV}}((k \oplus \text{opad}) || \text{H}_{h, \text{IV}}((k \oplus \text{ipad}) || r_{m'_1} || \dots || r_{m'_n})).$$

The above HMAC construction is a secure PRF, since its inner hash chain is a PRF even against backdooring adversaries. In particular, the first iteration in the inner chain (i.e. $h(\text{IV}, k \oplus$

¹We say that a function $f: \{0, 1\}^k \times \{0, 1\}^i \rightarrow \{0, 1\}^o$ for some constant s with $k < s \cdot o$ is an s -query weak PRF if $f(k, \cdot)$ (under a secret key k) is indistinguishable from a random function when evaluated at s independent known random inputs. This notion is weaker than a (regular) weak PRF where we require indistinguishability for polynomially many random inputs.

$\text{ipad}))$ is computationally indistinguishable from a uniformly-distributed random string, assuming as a weak dual PRF [7]. This guarantees that the first chaining value is pseudorandom and hence can be used as a “good” key in the RC construction. The same argument applies for the first chaining value in the outer chain. The last iteration in HMAC receives as input from both chains a pseudorandom input, and hence the output is still pseudorandom and thus HMAC is secure.

V. IMMUNIZATION OF KEY DERIVATION FUNCTIONS

The above transformation from a weak PRF to a full-fledged PRF can be expensive in terms of the number of compression function evaluations, which depends on the parameter s . Here we argue that for key derivation functions based on hash functions, in particular HKDF based on HMAC, there exists a simpler solution.

A. The Approach for HKDF

The HMAC-based key derivation function HKDF [29], [31] consists of two steps: an extraction step to smooth the entropy in some input key material like a Diffie-Hellman key, and an expansion step where sufficient key material is generated. The extraction step may use some public salt extsalt (if not present then it is set to 0) and produces a pseudorandom key PRK from the input key material IKM. The expand step takes the key PRK, some context information info like a transcript in a key exchange model, and the requested output length len (in octets). It iterates HMAC on PRK, the previous value, info , and a counter to generate sufficient key material. Formally,

$$\begin{aligned} \text{PRK} &\leftarrow \text{Extract}(\text{extsalt}, \text{IKM}) = \text{HMAC}(\text{extsalt}, \text{IKM}) \\ k &= k_1 || k_2 || \dots \leftarrow \text{Expand}(\text{PRK}, \text{info}, \text{len}) \end{aligned}$$

where $k_0 = \epsilon$, i.e., is the empty string, and $k_i = \text{HMAC}(\text{PRK}, k_{i-1} || \text{info} || i)$, with the counter value i being encoded as an octet. The last key part in the output may be truncated to match the requested output length.

Immunizing HKDF boils down to hardening HMAC and therefore the round function h . The security of HKDF relies on the pseudorandomness of h , which does not hold for backdoored functions according to our attacks on HMAC which we will describe in Section VII-B. As argued in the previous section, assuming that h is still a weak PRF in the presence of a backdoor appears to be more reasonable. Hence, our goal is to tweak HKDF to base its security on h to be a weak PRF.

We use the idea of Halevi and Krawczyk [28] to strengthen hash-and-sign schemes via input randomization. They propose to pick a fresh random string r with each signature generation and then compute the hash as $H((m_1 \oplus r) || \dots || (m_n \oplus r))$, XORing the random string to each message block. This alleviates the necessary assumption for the compression function h from collision resistance to some kind of second-preimage resistance. We stress that this strategy does not work to immunize hash functions against backdoors, as our attacks in Section VII-A1 show that a backdoored compression function would even allow to break second preimage resistance. In fact,

one can show that a backdooring adversary can still break the randomized hash-and-sign scheme.

The idea of Halevi and Krawczyk does apply, nonetheless, in the case of HMAC *when used as a key derivation function*, if we allow for a random value salt in the computation. Suppose that, when computing keying material, one is allowed to pick a random string salt of b bits. Then, instead of using the compression function h in the HMAC computations, we use the function $h^{\text{salt}}(x, y) = h(x, y \oplus \text{salt})$. Note that this means that we add salt to each input in each of the iteration steps.² When outputting the key material in the computation one can also return the value salt. The salt sometimes even needs to be published, e.g., in a key exchange protocol where the other party, too, should be able to derive the same key.

The downside of our construction is that each HMAC call in the expansion requires a fresh salt. However, usually only a few iterations in the expansion step are required. For example, the cipher suite AES_256_CBC_SHA256 in TLS 1.2 requires 128 key bytes such that four iterations, each with 256 bits output, suffice.

B. Security of Salted Key Derivation

To define security of the salted key derivation functions we adopt the approach of Krawczyk [29], demanding that the key derivation function provides pseudorandom outputs even when the adversary can ask to see derived keys on different information info. Since we use a fresh salt for each KDF call, we can even allow the adversary to query the same information info multiple times and still demand indistinguishability from fresh random key material.

In the security experiment below we again assume that we have a PPT generator $\text{BDKDFGen}(1^\lambda)$ which outputs a key derivation function KDF and possibly a backdoor bk. We assume that the function KDF takes two inputs, a context information info and a length input len, and returns a random value salt (of b bits) and keying material of len bits. To claim indistinguishability from random we consider an oracle $\mathcal{S}(\cdot)$ which on any input pair (info, len) returns a fresh random value salt and a random string k of len bits. Clearly, the adversary cannot ask its two oracles about the same input info, or else distinguishing the cases would be trivial. The salted-KDF-advantage of an adversary $\mathcal{A}$, optionally using the backdoor bk, is then defined by:

$$\begin{aligned} \text{Adv}_{\text{BDKDFGen}, \mathcal{A}, b}^{\text{skdf}}(\lambda) := & \left| \Pr \left[\mathcal{A}^{\text{KDF}(k, \cdot, \cdot), \text{KDF}(k, \cdot, \cdot)}(1^\lambda, [\text{bk}]_b) = 1 \mid \right. \right. \\ & (\text{KDF}, \text{bk}) \xleftarrow{\$} \text{BDKDFGen}(1^\lambda), k \xleftarrow{\$} \{0, 1\}^k \\ & \left. - \Pr \left[\mathcal{A}^{\text{KDF}(k, \cdot, \cdot), \mathcal{S}(\cdot, \cdot)}(1^\lambda, [\text{bk}]_b) = 1 \mid \right. \right. \\ & (\text{KDF}, \text{bk}) \xleftarrow{\$} \text{BDKDFGen}(1^\lambda), k \xleftarrow{\$} \{0, 1\}^k \left. \right] \right|, \end{aligned}$$

where the probability is over the choices of (KDF, bk) , k , the oracles answers, and $\mathcal{A}$'s coin tosses.

²As pointed out by Halevi and Krawczyk in [28] this also means that the padded message for the hash computation is masked with the random salt.

C. HKDF Expansion based on NMAC

We first discuss the case of expansion being based on NMAC instead of HMAC and argue afterwards that the result can be lifted to HMAC and the extraction step, making some additional assumptions. Recall that there are two differences between NMAC and its “practical cousin” HMAC. First, NMAC takes two independent keys $k_{\text{in}}, k_{\text{out}} \in \{0, 1\}^\ell$ instead of using correlated keys $k \oplus \text{ipad}, k \oplus \text{opad}$ as in HMAC. Second, the keys in NMAC are used directly as a substitute for the initialization vector IV, instead of making an extra iteration to first compute $h(\text{IV}, k \oplus \text{ipad})$ resp. $h(\text{IV}, k \oplus \text{opad})$ as in HMAC.

Let $\text{sNMAC}((k_{\text{in}}, k_{\text{out}}), \cdot)$ (for salted NMAC) be the probabilistic algorithm which, on being called, picks a fresh salt $\xleftarrow{\$} \{0, 1\}^b$ and then computes NMAC on keys $k_{\text{in}}, k_{\text{out}}$ for the salted compression function h^{salt} . It outputs the result of the computation together with the salt. By the construction of Expand we can assume that the adversary in the salted-KDF experiment only queries the key derivation function for length values $\text{len} = \ell$ equal to the output size of NMAC. This would already allow the adversary to assemble the full key material by sequentially making the corresponding queries.

Theorem V.1. *Let BDPRFGen be a backdoored PRF generator. Then sNMAC is a secure salted KDF, i.e., for any adversary $\mathcal{A}$ against sNMAC we obtain an adversary $\mathcal{B}$ against the weak PRF property with*

$$\text{Adv}_{\text{sNMAC}, \mathcal{A}, b}^{\text{skdf}}(\lambda) \leq 2nq \cdot \text{Adv}_{\text{BDPRFGen}, \mathcal{B}, b}^{w\text{PRF}}(\lambda)$$

where B is the maximal number of message blocks and each of them has at most q key derivation queries, and $n := B + 2 \cdot \lceil \ell/b \rceil + 3$. Furthermore the run times of $\mathcal{A}$ and $\mathcal{B}$ are essentially the same.

The proof idea is that each computation of sNMAC starts with an evaluation of the backdoored compression function for $x_2 = h(k_{\text{in}}, y_1 \oplus \text{salt})$, where y_1 is the first input block according to the adversary's query and salt is a fresh random value, picked independently after y_1 has been determined. This means that the input pair is random, such that we can conclude by the weak pseudorandomness that the output value x_2 looks random, too. The argument then applies to the next iteration step as well, since the next input $(x_2, y_2 \oplus \text{salt})$ to the compression function is (indistinguishable from) random. The approach can be set forth to show that all final answers in the computations look random, where the formal way to show this is via a hybrid argument. Since we pick a fresh salt in each computation, the result also holds for multiple queries. The formal proof can be found in the full version of the paper [25].

D. Lifting the Result to HKDF

To extend the above argument to also cover the extraction step we need to assume, as in the original security proof of HMAC [7], that the compression function h is a weak dual PRF. This means that $h(\cdot, \text{IKM})$ is weakly pseudorandom for the input keying material IKM (with sufficient entropy). This

appears to be a widely accepted assumption, but in our case this should also hold for backdoored compression functions. With a similar argument as in the wPRF case we can argue that a weak dual PRF remains secure (for fixed extraction salt extsalt) even when having a backdoor, or else one can again construct a public-key encryption scheme. The argument is as before, putting extsalt as part of the public-key and using the backdoor to distinguish random values (for encryptions of 0's) from $h(\text{extsalt}, \text{IKM})$ values (for encryptions of 1's, where the sender chooses IKM).

Similarly, we need to argue that using $k_{\text{in}} = h(\text{IV}, k \oplus \text{ipad})$ and $k_{\text{out}} = h(\text{IV}, k \oplus \text{opad})$ in the HMAC computation, instead of random values $k_{\text{in}}, k_{\text{out}}$ as in NMAC, does not endanger the security, even for backdoored h . The argument that the backdoored case should not make a difference is as before: pseudorandomness of the (correlated values) $h(\text{IV}, k \oplus \text{ipad})$ and $h(\text{IV}, k \oplus \text{opad})$ should also hold in the backdoored case, unless the backdoor already embeds a public-key encryption scheme.

VI. IMMUNIZATION OF TLS-LIKE KEY EXCHANGE

Once we have immunized HMAC and HKDF the next question is how we can use these building blocks in higher-level protocols to make them backdoor-resistant. We discuss this here briefly for the case of the TLS 1.3 protocol (in version draft-28 [39]), and especially for the pre-shared key (PSK) mode. The PSK mode covers the case in which client and server already hold a shared key and do not need to run a Diffie-Hellman key exchange sub-protocol; immunizing the latter would be beyond our work's scope. The PSK protocol only relies on a cryptographic hash function, but used in different contexts: as a collision-resistant hash function, as a MAC via HMAC, and as a key derivation function via HKDF.

A. Pre-Shared Key Mode of TLS 1.3

The PSK mode is displayed in Figure 3. We follow the presentation in [22], [23]. In the protocol the client starts with the `ClientHello` message, containing a nonce r_c , and specifies identifiers for shared keys via the `ClientPreSharedKey` message. The server replies with the `ServerHello` message, also containing a nonce r_s , and the choice of key identifier in `ServerPreSharedKey`. The server then starts deriving keys via HKDF on the pre-shared key PSK and the transcript hashes. It sends the encrypted extension information `{EncryptedExtensions}`. The server also computes a finished message `ServerFinished` which is an HMAC over the derived keys and the transcript hash. The client subsequently computes the keys, checks the HMAC, and sends its finished message `ClientFinished`. Both parties once more use HKDF and transcript hashes to derive the shared session key.

B. Towards Immunizing the PSK Mode

Not surprisingly, we are not able to show that the PSK mode of TLS 1.3, as is, can be immunized against backdoors. There are both security-related as well as functional reasons. In terms of security, the main problem is that the protocol crucially relies

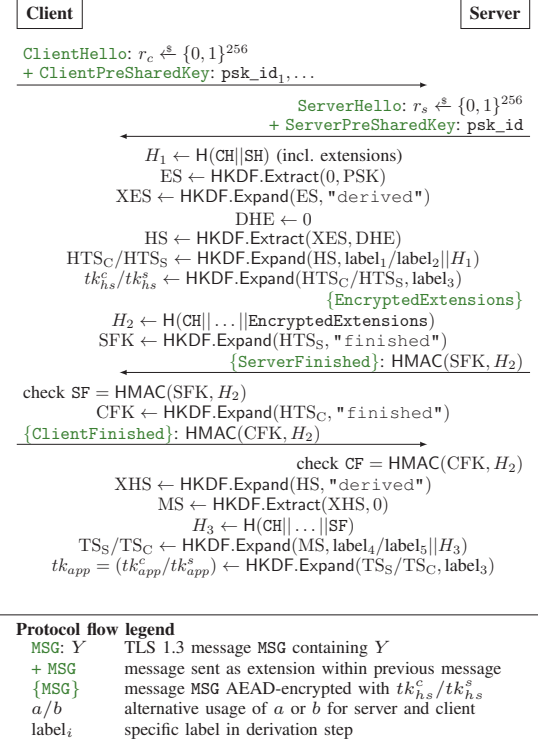


Figure 3: The TLS 1.3 draft-28 [39] PSK handshake protocol.

on the collision-resistance of the hash function to compute the transcript hashes. As we discuss in the next section, planting backdoors in collision-resistant hash functions is rather easy, such that we may not get immunity for the given protocol. Fortunately, the transcript hashes are only used to enable the parties to store the intermediate hash values instead of the entire transcript. In terms of security, one can easily forgo the transcript hashes and feed the full transcript into the immunized version of HMAC resp. HKDF.

Another obstacle to use our immunization strategy via salting of HKDF is that the salt needs to be picked independently of the input to the hash function. This can only be done by the party which evaluates the hash function next, e.g., when the server computes

$$\text{HTS}_C/\text{HTS}_S \leftarrow \text{HKDF.Expand}(\text{HS}, \text{label}_1/\text{label}_2 || H_1)$$

over the transcript hash $H_1 \leftarrow H(\text{CH} || \text{SH})$, or rather the full transcript $H_1 \leftarrow \text{CH} || \text{SH}$, to send the encrypted `{EncryptedExtensions}` message, then the entire input is only determined when the server is deriving the keys. The same holds on the client side for the finished message key CFK. Hence, we require that both parties at some point pick a random salt in a trustworthy way and therefore can only cover “backdooring” attacks against outsiders, eavesdropping on the communication. Still, we preserve active security against adversaries which cannot tamper with the cryptographic primitives.

Another problem with TLS 1.3 in its current form is that it is not clear how to embed the salt in the protocol flow. The extensions currently do not offer a variable field for this. Hence, one would need to change the specification to enable the inclusion of such extra data, as well as the algorithm specifiers to capture the salted versions.

With all the modifications above, one obtains a PSK mode which only relies on the backdoor-resistant modified primitives HMAC and HKDF. We omit a formal analysis as it would require to define security of key exchange protocols and is beyond the scope here.

VII. BACKDOORED MERKLE-DAMGÅRD-BASED HASH FUNCTIONS AND HMAC

In this section, we turn our attention to demonstrating the feasibility of embedding a backdoor in ordinary hash functions, such that the adversary in possession of the backdoor is able to undermine the most crucial security properties of the hash function. At the same time, the hash function retains all those security properties against adversaries without knowledge of the backdoor key. Moreover, the specification only uses symmetric-key primitives and is secure against reverse engineering attempts, in that it cryptographically hides the backdoor key. For our construction we drew inspiration from many-to-one trapdoor one-way functions with an exponential preimage size as studied by Bellare et al. [11]. They show that such trapdoor one-way functions can be built from one-way functions and hence building secure public-key encryption from them is hard.

We construct a backdoored Merkle-Damgård-based hash function by iterating a backdoored compression function, which behaves like a secure compression function unless the backdoor is triggered by a special key as part of the input message. The proposed construction might seem rather simple but demonstrates how easy it is to embed a backdoor into the building block such that its knowledge enables an adversary to easily violate the hash function's security properties. We also investigate whether our construction has any weakening impact when used in HMAC (where HMAC is used as is, without our immunization modifications). Unfortunately, we have to answer this in the affirmative and show that even though HMAC uses a secret key, it is not secure, since the adversary has full control over the input message and can trigger the backdoor for the underlying hash function.

In the full version of the paper [25], we provide a similar result for sponge-based hash functions. Namely, we discuss backdooring the underlying permutation of the sponge-based hash function construction and show that such a backdoored construction violates the security properties by describing attacks.

A. Backdoored MD-based Hash Functions

Let $h: \{0, 1\}^\ell \times \{0, 1\}^b \rightarrow \{0, 1\}^\ell$ be a collision-resistant compression function with $b \geq 2\ell$. The algorithm BDHGen creates a backdoored compression function $\tilde{h}: \{0, 1\}^\ell \times \{0, 1\}^b \rightarrow \{0, 1\}^\ell$ and its backdoor key $\text{bk} \in \{0, 1\}^\ell$, as

defined below. In order for $\tilde{h}$ to be collision-resistant against adversaries without knowledge of the backdoor, $\tilde{h}$ behaves essentially like h , unless the backdoor is triggered. The backdoor bk is defined in a way that $h(\text{bk}, c) = \beta$, for fixed values $c \in \{0, 1\}^b$ and $\beta \in \{0, 1\}^\ell$. The values bk and c can for instance be chosen randomly by the malicious designer of $\tilde{h}$, from which β is determined.

compression function :

$$\tilde{h}: \{0, 1\}^\ell \times \{0, 1\}^b \rightarrow \{0, 1\}^\ell$$

$$\tilde{h}(x, y) := \begin{cases} y_{[\ell, 2\ell-1]}, & \text{if } h(y_{[0, \ell-1]}, c) = \beta \\ h(x, y), & \text{otherwise} \end{cases}$$

backdoor key :

random $\text{bk} \in \{0, 1\}^\ell$ and $c \in \{0, 1\}^b$
such that $h(\text{bk}, c) = \beta$

Figure 4: Backdoored compression function $\tilde{h}$ and its backdoor key bk , built from any compression function $h: \{0, 1\}^\ell \times \{0, 1\}^b \rightarrow \{0, 1\}^\ell$ with $b \geq 2\ell$.

The backdoor key is a particular point bk of admissible inputs of h . The malicious compression function $\tilde{h}$ simply evaluates h , except if it detects a backdoor key in the input. Let us take a closer look at the definition of $\tilde{h}$ from Figure 4. In the first case, the first ℓ bits of the processed b -bit block correspond to the backdoor value bk (or another preimage of β under $h(\cdot, c)$), then $\tilde{h}$ sets the output to the next ℓ bits of that input string. This allows to program the output of this iteration since the message block can be thought of as being under adversarial control. For all other inputs, the second case of $\tilde{h}$ applies, which simply calls the underlying compression function h on the inputs. In the next section, we describe the attacks in more detail.

Our backdoor uses an if-then-else construct. Such constructs, or derivatives thereof, are often implicit in the design of round functions of hash algorithms for nonlinearity reasons. For instance, SHA-1 and SHA-2 use the function $\text{Ch}(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z)$ on 32-bit words in the round evaluations, implementing a bit-wise “if x_i then y_i ; else z_i ” simultaneously over words. In SHA-3 the χ operation $\chi(a, b, c) = c \oplus (\neg a \wedge b)$ on 64-bit words can be viewed to implement “if a_i then c_i else $b_i \oplus c_i$ ” for each bit in the words.

We stress, however, that we are not claiming that SHA-1, SHA-2, or SHA-3 actually have backdoors. In particular, embedding our backdoors would introduce additional complications since one has less control over the inputs when the operations Ch and χ are applied in the iterations of the round functions. Our construction merely demonstrates that incorporating hidden backdoors is possible in principle, and that only mild operations are necessary to exploit the backdoor.

Proposition VII.1. *The compression function $\tilde{h}$ given in*

Figure 4 is collision resistant if the underlying h is collision resistant and $h(\cdot, c)$ is preimage resistant (for parameter ℓ) for randomly chosen $c \xleftarrow{\$} \{0, 1\}^b$.

The idea is that a collision finder can only take advantage of the embedded case if it finds a preimage for β for $h(\cdot, c)$. Else it needs to find a collision from scratch.

Proof. Suppose that a PPT adversary $\mathcal{A}$ finds collisions for $\tilde{h}$ with some probability ε , i.e., outputs $(x, y) \neq (x', y')$, such that $\tilde{h}(x, y) = \tilde{h}(x', y')$. We make a case distinction:

- Assume that $h(y_{[0, \ell-1]}, c) = \beta$ or $h(y'_{[0, \ell-1]}, c) = \beta$. Then it is straightforward to build an adversary against the preimage resistance of $h(\cdot, c)$, since $y_{[0, \ell-1]}$ resp. $y'_{[0, \ell-1]}$ constitutes a preimage for β .
- According to the other case we thus have $\tilde{h}(x, y) = h(x, y) = h(x', y') = \tilde{h}(x', y')$ for $(x, y) \neq (x', y')$. This, however, contradicts the collision resistance of h .

In summary, an adversary $\mathcal{A}$ successfully attacking collision resistance of $\tilde{h}$ can be used to build an adversary that can either find preimages for $h(\cdot, c)$ or find collisions under h (in the same time). Hence, $\mathcal{A}$'s success probability is bounded by the sum of these cases. $\square$

With a similar argument we can show that the same holds for the other properties:

Proposition VII.2. *The compression function $\tilde{h}$ given in Figure 4 is preimage resistant if the underlying h is preimage resistant for parameter $\ell + b$ and if $h(\cdot, c)$ is preimage resistant for parameter ℓ for randomly chosen $c \xleftarrow{\$} \{0, 1\}^b$.*

As in the proof for collision resistance this holds as an adversary $\mathcal{A}$ against preimage resistance either needs to find a preimage for parameter ℓ (i.e., a backdoor key), or under the original function h for parameter $\ell + b$.

Proposition VII.3. *The compression function $\tilde{h}$ given in Figure 4 is second-preimage resistant for parameter $\ell + b$ if the underlying h is second-preimage resistant for $\ell + b$ and $h(\cdot, c)$ is preimage resistant for parameter ℓ for randomly chosen $c \xleftarrow{\$} \{0, 1\}^b$.*

Next we build from $\tilde{h}$ a backdoored hash function $\mathcal{H}_h^{\text{md}} := (\text{KGen}^{\text{md}}, \text{H}_h^{\text{md}})$ using the standard Merkle-Damgård domain extender, which iterates the backdoored compression function $\tilde{h}$. Intuitively, with the backdoor key an adversary can trigger one or more iterations of the compression function to land in a weak mode, and abuse it to break the hash function, i.e., find collisions, preimages, and second preimages. In the following section we discuss the attacks in detail.

Remark. Although it is hard to find the backdoor itself, the fact that the above construction is backdoored is fairly obvious. However, techniques such as practical obfuscation may be employed to better hide the (implicit) if-clause and the constants used for backdooring. The complexity of permutations and compression functions used in practice makes detection even harder. Alternatively, the attacker may also aim at the implementation and insert malicious code in a

software update. The new backdoored implementation remains indistinguishable from its specification via black-box testing.

1) Attack Description: In this section we describe how a backdoor key bk as described in Figure 4 can be used to break collision resistance, preimage resistance and second-preimage resistance of the above backdoored hash function $\mathcal{H}_h^{\text{md}}$. Precise definitions of these properties were given in Section II-B. For finding collisions and preimages, the adversary does not need to actually evaluate the hash function at all. For finding second preimages, the adversary evaluates the hash function once and only for computing the digest of the given message. Here, we consider the compact length padding defined in Section II-A1. However, we remark that backdoored compression functions, similar to the one that we discuss below, do exist for other padding functions.

FINDING COLLISIONS. The adversary's goal is to find two messages $m, m' \in \{0, 1\}^{\leq 2^p}$, that will be mapped to the same digest. We argue that any two messages m and m' of the same length that have a common suffix starting with bk and an arbitrary prefix of the same size, which is a multiple of the block size b , collide. It immediately follows from the exponential size of the message space that the backdoor enables finding an exponential number of collisions. More precisely, for all prefixes $v, v' \in (\{0, 1\}^b)^+$ with $|v| = |v'|$ and suffixes $w \in \{0, 1\}^*$ the colliding messages are:

$$m = v \parallel \text{bk} \parallel w \quad \text{and} \quad m' = v' \parallel \text{bk} \parallel w.$$

Observe that, after padding, both messages contain the backdoor bk at the beginning of the same block. This triggers the “bad” case of the backdoored compression function, which then outputs the following ℓ bits of the message, ignoring the intermediate value x obtained from the previous iteration. Since the messages do not differ after the backdoor and neither does the padding of the equal length messages, their digests eventually coincide.

In a similar attack, the adversary may be able to encode an expected digest $d \in \{0, 1\}^\ell$ directly in the messages. That is, for all prefixes $v, v' \in (\{0, 1\}^b)^+$ of the same size (i.e., $|v| = |v'|$), and short suffixes $u, u' \in \{0, 1\}^{\leq b-2\ell-p-1}$ the following messages are both mapped to the same value by the backdoored hash function:

$$m = v \parallel \text{bk} \parallel d \parallel u \quad \text{and} \quad m' = v' \parallel \text{bk} \parallel d \parallel u'.$$

If $b \geq 2\ell + p + 1$, the lengths of the messages fit in the same block as bk , d , and u resp. u' . Hence, the encoded length $\text{binary}(|m|, p)$ resp. $\text{binary}(|m'|, p)$ is simply ignored by the backdoored compression function and d is output as the digest for both. Here, the messages do not even have to be of the same size. However, if $2\ell \leq b < 2\ell + p + 1$ the binary encoding of the message lengths cannot be included in the same block as bk and d , hence, the above messages will collide (mapped to a different digest than d) if their last blocks after padding (in particular their lengths) are equal.

FINDING PREIMAGES. Here we describe how an adversary can use the backdoor to find a preimage for a random

digest $d \in \{0,1\}^\ell$. We distinguish two cases based on the length of message blocks. First we consider $b \geq 2\ell + p + 1$, where the backdoor key, the challenge digest, and the padded message length can all fit in a single block. Second, we consider $2\ell \leq b < 2\ell + p + 1$, where at least the backdoor key and the message length can fit into the same block, however not together with the challenge digest.

If $b \geq 2\ell + p + 1$, the adversary can encode the digest d in the message directly after the backdoor key bk , while still leaving space in the last block for the message length to be encoded in by the padding. More precisely, for all prefixes $v \in (\{0,1\}^b)^+$ and all short suffixes $u \in \{0,1\}^{\leq b-2\ell-p-1}$ the following message m is a valid preimage for d :

$$m = v||\text{bk}||d||u.$$

Here, after the first malicious case of the compression function is triggered by bk , the digest d will be output, and the rest of the last block, containing the encoded length is simply ignored.

The attack is slightly more subtle in case of $2\ell \leq b < 2\ell + p + 1$. However, one can still trigger the backdoor in the last message block, causing the next ℓ bits to be output by the backdoored compression function. Consider the message $m = v||\text{bk}||u$ now with $u \in \{0,1\}^{\leq b-\ell-p-1}$. If the ℓ bits of the padded message immediately following the backdoor key bk , i.e., u and a prefix of $\text{lpad}(m, b, p)$, correspond to the challenge digest d , then m is a valid preimage.

FINDING SECOND-PREIMAGES. Finding second preimages is very similar to finding preimages. In fact the adversary can perform the above attacks to find a second preimage m' for a given message m , after setting $d = H_{h,IV}^{\text{md}}(m)$. Note that since the adversary can find an exponential number of preimages by choosing different prefixes and suffixes, she can easily find a preimage m' of d that is not equal to the original message m .

2) *Exposure of Backdoor Key:* As discussed, a backdoor can enable adversaries to break security of a hash function. The same backdoored construction is unexploitable by an adversary who does not know the backdoor key. Attempts at detecting a potential backdoor via black-box testing or finding the backdoor key by reverse engineering the code may easily fail.

However, observe that every collision, preimage, or second preimage found using the backdoor key, encodes the backdoor key in the message. Therefore, using the backdoor may put the adversary in risk of being exposed. It is unclear whether constructions of backdoored compression functions are possible that do not expose their backdoor key in adversarial inputs and do not rely on indistinguishability obfuscation to hide a secret key in the compression function and use it to internally decrypt malicious triggers.

B. Backdoored HMAC

In this section, we discuss that building HMAC upon the backdoored Merkle-Damgård hash function $\mathcal{H}_h^{\text{md}}$ of Section VII-A yields a backdoored HMAC scheme, which is

easily forgeable using the backdoor key. More precisely, the backdoored HMAC scheme is defined as $\mathcal{HMAC}_{\tilde{h}} := (\text{KGen}, \text{HMAC}_{\tilde{h}})$. However, note that $\tilde{h}$ is still a PRF against adversaries that do not know a backdoor, as we prove below. Therefore, the resulting HMAC construction $\mathcal{HMAC}_{\tilde{h}}$ is also a PRF against such adversaries.

Lemma VII.4. *The compression function $\tilde{h}$ from Section VII is a PRF if the underlying function h is a PRF, and if $h(\cdot, c)$ is preimage resistant for parameter ℓ for random $c \xleftarrow{\$} \{0,1\}^b$.*

Proof. Assume that there exist an adversary $\mathcal{A}$ with a non-negligible advantage $\text{Adv}_{h,A,0}^{\text{PRF}}(\lambda)$ in distinguishing $\tilde{h} : \{0,1\}^\ell \times \{0,1\}^b \rightarrow \{0,1\}^\ell$ from a random function with the same domain and range. We use $\mathcal{A}$ to build an adversary $\mathcal{B}$ against the PRF-security of h as follows. By definition $\mathcal{B}$ gets access to an oracle which either implements $h(k, \cdot)$, for a random key k , or a truly random function $f_{\$}$.

Initially, $\mathcal{B}$ picks random values bk, c and computes β as $\beta = h(\text{bk}, c)$. Upon receiving a query $y \in \{0,1\}^b$ from $\mathcal{A}$, our new adversary $\mathcal{B}$ simply forwards this query to its oracle and returns the answer unless $h(y_{[0,\ell-1]}, c) = \beta$ is met, in which case $y_{[\ell,2\ell-1]}$ is returned. When the adversary $\mathcal{A}$ terminates with output b , then so does $\mathcal{B}$.

For the analysis note that, in case that $\mathcal{B}$ is communicating with the oracle h , the only difference in the answers handed to $\mathcal{A}$ lie in the exceptional case that $h(y_{[0,\ell-1]}, c) = \beta$. This means that we can compute a preimage of β under $h(\cdot, c)$ with the help of $\mathcal{A}$'s queries, which straightforwardly leads to a contradiction to the preimage resistance of h (via the construction of some algorithm $\mathcal{C}$ against preimage resistance derived from $\mathcal{A}$ resp. derived by a pure guessing strategy) and thus have small probability only. Hence,

$$\Pr[\mathcal{B}^{h(k,\cdot)}(1^\lambda) = 1] \geq \Pr[\mathcal{A}^{\tilde{h}(k,\cdot)}(1^\lambda) = 1] - \text{Adv}_{h,C,0}^{\text{PR},\ell}(\lambda).$$

For a truly random function oracle the behavior of $\mathcal{A}$ and $\mathcal{B}$ are identical. Therefore,

$$\text{Adv}_{h,B,0}^{\text{PRF}}(\lambda) \geq \text{Adv}_{h,A,0}^{\text{PRF}}(\lambda) - \text{Adv}_{h,C,0}^{\text{PR},\ell}(\lambda)$$

This, however, contradicts the PRF-security (or the preimage resistance) of h . $\square$

Note that it is also unlikely that the HMAC case of first computing $\tilde{h}(\text{IV}, k \oplus \text{ipad})$ resp. $\tilde{h}(\text{IV}, k \oplus \text{opad})$ triggers the exceptional branch. The reason is that this could only happen if the key parts constituted a preimage of the backdoor value β .

1) *Attack Description:* Recall that the backdoor bk , defined in Figure 4, allows an adversary to find collisions for the underlying hash function $\mathcal{H}_h^{\text{md}}$. Finding collisions for the inner hash chain of the backdoored HMAC construction is precisely what makes forging MAC tags possible. First, the adversary queries $\text{HMAC}_{\tilde{h}}$ on a message $m = v||\text{bk}||w$, where $v \in (\{0,1\}^b)^+$ and $w \in \{0,1\}^*$. After receiving the corresponding tag t , the adversary returns the pair $(m^*, t) = (v' || \text{bk} || w, t)$ as a forgery, where $v' \in (\{0,1\}^b)^+$, $v \neq v'$, and $|v| = |v'|$.

As discussed in Section VII-A1, the messages m and m^* lead to collisions in H_h^{md} . Since their prefixes v and v' of equal

length can be arbitrary, they can in particular start with a block of b -bits equal to $k \oplus \text{ipad}$. Hence, it is easy to see that after the messages are prepended by $k \oplus \text{ipad}$, they still lead to a collision in the inner hash chain of $\text{HMAC}_{\tilde{h}}$. Since the outer chain is equal for all messages, m and m^* both have the same tag t . Putting differently, HMAC is not backdoor-resilient just because it uses a secret key. In summary, since an adversary holding a backdoor can forge a tag for a new message, $\text{HMAC}_{\tilde{h}}$ is forgeable and hence not pseudorandom.

VIII. RELATED WORK

Techniques of mass-surveillance in the kleptographic setting can be roughly divided into backdooring cryptosystems and algorithm-substitution attacks (ASAs). A backdoor targets the design and/or the public parameters of a primitive, while ASAs target the implementation.

In the realm of backdoored hash functions, Albertini et al. [1] investigate backdoored hash functions designed by abusing the freedom of choice in selecting the round constants. They illustrate the possibility of malicious hash function designs by providing a tailored version of SHA-1, such that two certain colliding messages that are adaptively chosen with the malicious round constants during the design are found with an approximate complexity of 2^{48} . In comparison, the complexity of finding collisions for the standard SHA-1 function is believed to be over 2^{63} . Similarly, Aumasson [4] presents a malicious version of the SHA-3 competition's finalist BLAKE where the attacker adaptively modifies operators in the finalization function to find collisions. Furthermore, Morawiecki [36] proposes a malicious variant of Keccak (the winner of the SHA-3 competition) and AlTawy and Youssef [2] present a backdoored version of Streebog which is a Russian cryptographic standard. Both papers introduce modified round constants generating collisions using differential cryptanalysis.

Inspired by the Dual-EC tragedy, Dodis et al. [21] initiated the formal study of backdoored pseudorandom number generators, proving their equivalence to public-key encryption and discussing immunization strategies. Their notion is extended by Degabriele et al. [20] in order to investigate stronger "backdoorability" of forward-secure pseudorandom generators and pseudorandom number generators with refreshed states. Bernstein et al. [14] analyzed the possibilities of maliciously standardized elliptic curves.

Bellare et al. [13] formalize algorithm-substitution attacks in the context of symmetric key encryption. They describe attacks, where subverted randomized encryption algorithms can leak the user's secret key subliminally and undetectably to the adversary. Understanding ASAs and possible detection and prevention mechanisms was followed by several work [3], [12], [19], [35], [40], [41], [26].

Notable works in the context of the Dual_EC_DRBG-related incidents are [16] and [17] by Checkoway et al. that provide a systematic analysis as well as a study on the practical exploitability of the backdoor.

IX. CONCLUSION

Developing immunization strategies with meaningful protection against the threat of maliciously designed cryptosystems is a challenging and non-trivial task. Relying on our observation that efficient weak pseudorandom functions, which do not contain public-key encryption, cannot be weakened by a backdoor, we gave solutions for immunizing potentially backdoored HMAC and HKDF constructions.

A natural open question is, whether immunizing publicly keyed hash functions under reasonable assumptions is possible. Since inputs of hash functions are under adversarial control and can be used to trigger malicious behavior, a generic solution via a priori transformation of the inputs to destroy its potentially malicious structure does not seem to exist. However, there may be immunization strategies that are specific to the applications of hash functions.

Not only is it important to immunize potentially backdoored hash functions, but in order to facilitate detection of backdoored functions in practice and design hash functions that inherently resist backdoors, it is also necessary to understand the various ways backdoors can be embedded in hash functions. Our construction of a backdoored hash function shows that it is mathematically feasible to embed a powerful backdoor (which is exclusive to the malicious designer) in a hash function, while not sacrificing efficiency. An important extension in this direction is to study different backdooring attempts that are less apparent and harder to detect even if they are potentially less powerful.

REFERENCES

- [1] A. Albertini, J.-P. Aumasson, M. Eichlseder, F. Mendel, and M. Schl fer. Malicious hashing: Eve's variant of SHA-1. In A. Joux and A. M. Youssef, editors, *SAC 2014*, volume 8781 of *LNCS*, pages 1–19. Springer, Heidelberg, Aug. 2014.
- [2] R. AlTawy and A. M. Youssef. Watch your constants: malicious streebog. *IET Information Security*, 9(6):328–333, 2015.
- [3] G. Ateniese, B. Magri, and D. Venturi. Subversion-resilient signature schemes. In I. Ray, N. Li, and C. Kruegel, editors, *ACM CCS 15*, pages 364–375. ACM Press, Oct. 2015.
- [4] J.-P. Aumasson. Eve's sha3 candidate: malicious hashing.
- [5] J. Ball, J. Borger, and G. Greenwald. Revealed: how US and UK spy agencies defeat internet privacy and security. <http://www.theguardian.com/world/2013/sep/05/nsa-gchq-encryption-codes-security>, Sept. 2013.
- [6] E. B. Barker and J. M. Kelsey. Sp 800-90a. recommendation for random number generation using deterministic random bit generators. Technical report, Gaithersburg, MD, United States, 2012.
- [7] M. Bellare. New proofs for NMAC and HMAC: Security without collision-resistance. In C. Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 602–619. Springer, Heidelberg, Aug. 2006.
- [8] M. Bellare. New proofs for NMAC and HMAC: Security without collision resistance. *Journal of Cryptology*, 28(4):844–878, Oct. 2015.
- [9] M. Bellare, R. Canetti, and H. Krawczyk. Keying hash functions for message authentication. In N. Kobitz, editor, *CRYPTO '96*, volume 1109 of *LNCS*, pages 1–15. Springer, Heidelberg, Aug. 1996.
- [10] M. Bellare, R. Canetti, and H. Krawczyk. Pseudorandom functions revisited: The cascade construction and its concrete security. In *37th FOCS*, pages 514–523. IEEE Computer Society Press, Oct. 1996.
- [11] M. Bellare, S. Halevi, A. Sahai, and S. P. Vadhan. Many-to-one trapdoor functions and their relation to public-key cryptosystems. In H. Krawczyk, editor, *CRYPTO '98*, volume 1462 of *LNCS*, pages 283–298. Springer, Heidelberg, Aug. 1998.

- [12] M. Bellare, J. Jaeger, and D. Kane. Mass-surveillance without the state: Strongly undetectable algorithm-substitution attacks. In I. Ray, N. Li, and C. Kruegel, editors, *ACM CCS 15*, pages 1431–1440. ACM Press, Oct. 2015.
- [13] M. Bellare, K. G. Paterson, and P. Rogaway. Security of symmetric encryption against mass surveillance. In J. A. Garay and R. Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 1–19. Springer, Heidelberg, Aug. 2014.
- [14] D. J. Bernstein, T. Chou, C. Chuengsatiansup, A. Hülsing, E. Lambooi, T. Lange, R. Niederhagen, and C. van Vredendaal. How to manipulate curve standards: A white paper for the black hat <http://bada55.cr.yp.to>. In L. Chen and S. Matsuo, editors, *Security Standardisation Research - Second International Conference, SSR 2015, Tokyo, Japan, December 15-16, 2015, Proceedings*, volume 9497 of *Lecture Notes in Computer Science*, pages 109–139. Springer, 2015.
- [15] D. J. Bernstein, T. Lange, and R. Niederhagen. Dual EC: A standardized back door. In P. Y. A. Ryan, D. Naccache, and J. Quisquater, editors, *The New Codebreakers - Essays Dedicated to David Kahn on the Occasion of His 85th Birthday*, volume 9100 of *Lecture Notes in Computer Science*, pages 256–281. Springer, 2016.
- [16] S. Checkoway, J. Maskiewicz, C. Garman, J. Fried, S. Cohn, M. Green, N. Heninger, R.-P. Weinmann, E. Rescorla, and H. Shacham. A systematic analysis of the juniper dual EC incident. In E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, editors, *ACM CCS 16*, pages 468–479. ACM Press, Oct. 2016.
- [17] S. Checkoway, R. Niederhagen, A. Everspaugh, M. Green, T. Lange, T. Ristenpart, D. J. Bernstein, J. Maskiewicz, H. Shacham, and M. Fredrikson. On the practical exploitability of dual EC in TLS implementations. In K. Fu and J. Jung, editors, *Proceedings of the 23rd USENIX Security Symposium, San Diego, CA, USA, August 20-22, 2014.*, pages 319–335. USENIX Association, 2014.
- [18] I. Damgård. A design principle for hash functions. In G. Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 416–427. Springer, Heidelberg, Aug. 1990.
- [19] J. P. Degabriele, P. Farshim, and B. Poettering. A more cautious approach to security against mass surveillance. In G. Leander, editor, *FSE 2015*, volume 9054 of *LNCS*, pages 579–598. Springer, Heidelberg, Mar. 2015.
- [20] J. P. Degabriele, K. G. Paterson, J. C. N. Schuldt, and J. Woodage. Backdoors in pseudorandom number generators: Possibility and impossibility results. In M. Robshaw and J. Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 403–432. Springer, Heidelberg, Aug. 2016.
- [21] Y. Dodis, C. Ganes, A. Golovnev, A. Juels, and T. Ristenpart. A formal treatment of backdoored pseudorandom generators. In E. Oswald and M. Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 101–126. Springer, Heidelberg, Apr. 2015.
- [22] B. Dowling, M. Fischlin, F. Günther, and D. Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol candidates. In I. Ray, N. Li, and C. Kruegel, editors, *ACM CCS 15*, pages 1197–1210. ACM Press, Oct. 2015.
- [23] B. Dowling, M. Fischlin, F. Günther, and D. Stebila. A cryptographic analysis of the TLS 1.3 draft-10 full and pre-shared key handshake protocol. Cryptology ePrint Archive, Report 2016/081, 2016. <http://eprint.iacr.org/2016/081>.
- [24] N. FIPS. 198: The keyed-hash message authentication code (hmac). *National Institute of Standards and Technology, Federal Information Processing Standards*, page 29, 2002.
- [25] M. Fischlin, C. Janson, and S. Mazaheri. Backdoored Hash Functions: immunizing HMAC and HKDF. Cryptology ePrint Archive, Report 2018/362, 2018. <http://eprint.iacr.org/2018/362>.
- [26] M. Fischlin and S. Mazaheri. Self-guarding cryptographic protocols against algorithm substitution attacks. Cryptology ePrint Archive, Report 2017/984, 2017. <http://eprint.iacr.org/2017/984>.
- [27] G. Greenwald. *No Place to Hide: Edward Snowden, the NSA, and the U.S. Surveillance State*. Metropolitan Books, USA, 2014.
- [28] S. Halevi and H. Krawczyk. Strengthening digital signatures via randomized hashing. In C. Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 41–59. Springer, Heidelberg, Aug. 2006.
- [29] H. Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In T. Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 631–648. Springer, Heidelberg, Aug. 2010.
- [30] H. Krawczyk, M. Bellare, and R. Canetti. Hmac: Keyed-hashing for message authentication. *RFC 2104*, 1997.
- [31] H. Krawczyk and P. Eronen. Hmac-based extract-and-expand key derivation function (hkdf). *RFC 5869*, 2010.
- [32] U. M. Maurer and J. Sjödin. A fast and key-efficient reduction of chosen-ciphertext to known-plaintext security. In M. Naor, editor, *EUROCRYPT 2007*, volume 4515 of *LNCS*, pages 498–516. Springer, Heidelberg, May 2007.
- [33] U. M. Maurer and S. Tessaro. Basing PRFs on constant-query weak PRFs: Minimizing assumptions for efficient symmetric cryptography. In J. Pieprzyk, editor, *ASIACRYPT 2008*, volume 5350 of *LNCS*, pages 161–178. Springer, Heidelberg, Dec. 2008.
- [34] R. C. Merkle. One way hash functions and DES. In G. Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 428–446. Springer, Heidelberg, Aug. 1990.
- [35] I. Mironov and N. Stephens-Davidowitz. Cryptographic reverse firewalls. In E. Oswald and M. Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 657–686. Springer, Heidelberg, Apr. 2015.
- [36] P. Morawiecki. Malicious Keccak. Cryptology ePrint Archive, Report 2015/1085, 2015. <http://eprint.iacr.org/2015/1085>.
- [37] M. Naor and O. Reingold. Synthesizers and their application to the parallel construction of pseudo-random functions. *J. Comput. Syst. Sci.*, 58(2):336–375, 1999.
- [38] K. Pietrzak and J. Sjödin. Weak pseudorandom functions in minicrypt. In L. Aceto, I. Damgård, L. A. Goldberg, M. M. Halldórsson, A. Ingólfssdóttir, and I. Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 423–436. Springer, Heidelberg, July 2008.
- [39] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3 – draft-ietf-tls-tls13-28. <https://tools.ietf.org/html/draft-ietf-tls-tls13-28>, Mar. 2018.
- [40] A. Russell, Q. Tang, M. Yung, and H.-S. Zhou. Clipping the power of kleptographic attacks. In J. H. Cheon and T. Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 34–64. Springer, Heidelberg, Dec. 2016.
- [41] A. Russell, Q. Tang, M. Yung, and H.-S. Zhou. Generic semantic security against a kleptographic adversary. In B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, editors, *ACM CCS 17*, pages 907–922. ACM Press, Oct. / Nov. 2017.
- [42] A. Young and M. Yung. The dark side of “black-box” cryptography, or: Should we trust capstone? In N. Kobitz, editor, *CRYPTO'96*, volume 1109 of *LNCS*, pages 89–103. Springer, Heidelberg, Aug. 1996.
- [43] A. Young and M. Yung. Kleptography: Using cryptography against cryptography. In W. Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 62–74. Springer, Heidelberg, May 1997.